

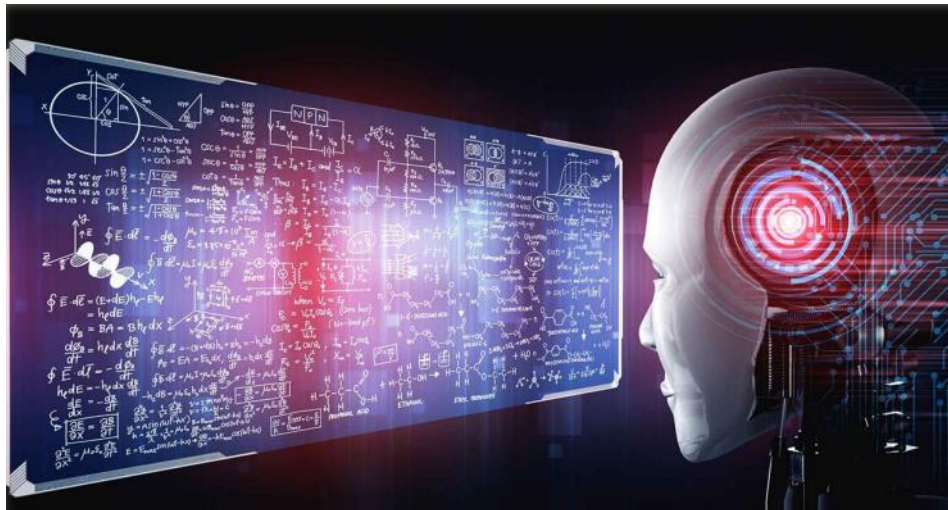
الجامعة العربية الأمريكية
ARAB AMERICAN UNIVERSITY



FACULTY OF ENGINEERING

ALGORITHMS ANALYSIS AND DESIGN

HUFFMAN ALGORITHM



DR. THAER THAHER

BY:

ZAID ARDAH

MAHMOUD AZIZA

SAMI AMARNEH

Data Compression Project Using Huffman Algorithm

Introduction:

In today's world, where we generate and share massive amounts of data every second, finding efficient ways to store and transmit information is more important than ever. When dealing with millions of lines of text, a key question arises: Can we shrink the data size without losing any information? That's where data compression comes in it helps reduce file sizes while keeping the content intact.

One of the most effective compression techniques is **Huffman's algorithm**. It takes advantage of patterns in the data by giving shorter codes to frequently used characters and longer codes to fewer common ones, making storage and transmission more efficient.

In this project, we'll explore Huffman's algorithm from both a theoretical and practical perspective. We'll break down how it works, design a system using it, and implement it in **C++**. Finally, we'll analyze its performance and evaluate how well it works in real-world scenarios.

Theoretical Background:

Data Compression: Data compression is all about making files smaller by removing unnecessary repetition or finding patterns in the data. It's a crucial technique used in many areas, from saving space on storage devices to speeding up data transfer over the internet. Whether it's text, images, videos, or audio, compression helps reduce file sizes without losing essential information, making everyday technology like streaming and file sharing more efficient.

ASCII Encoding: ASCII (American Standard Code for Information Interchange) is one of the earliest ways of encoding text on computers. It assigns a fixed-length binary code (7 or 8 bits) to each character, meaning every letter, number, or symbol takes up the same amount of space. The downside? It doesn't consider how often a character appears, so it's not the most efficient way to store text, especially in large files where some characters are used much more frequently than others.

Huffman Algorithm: In 1952, **David Huffman** came up with a smarter way to encode data: Huffman's algorithm. Instead of using the same number of bits for every character, it assigns **shorter codes to more common characters and longer codes to less frequent ones**. This results in smaller file sizes. The algorithm uses a **Greedy approach**, meaning it makes the best choice at each step to build an optimal solution. It works by repeatedly combining the two least frequent characters (or groups of characters) into a **prefix tree** until all elements are merged. This ensures the most efficient encoding possible, reducing the average code length and improving compression performance.

Methodology:

Steps to Build a Huffman Tree:

1. **Counting Character Frequency:**

First, we scan the text and count how often each character appears.

2. **Creating Nodes:**

For every unique character in the text, we create a node that stores the character and its frequency.

3. **Building the Huffman Tree:**

- We use a **Min Heap** (or priority queue) to keep track of the nodes with the smallest frequencies.
- At each step, the two least frequent nodes are merged into a new node, whose frequency is the sum of the two.
- This process is repeated until all nodes are combined into a single tree.

4. **Generating Huffman Codes:**

- Once the tree is built, we assign a unique binary code to each character by tracing its path from the root to the leaf.
- Moving **left** in the tree adds a "0", while moving **right** adds a "1".

Simplified Example:

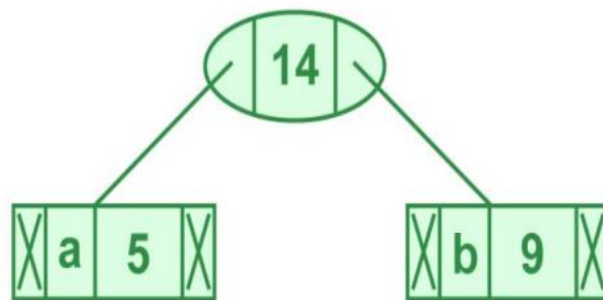
Consider the following example of a set of characters along with their respective frequencies:

Character	Frequency
A	5
B	9
C	12
D	13
E	16
F	45

Steps to solve:

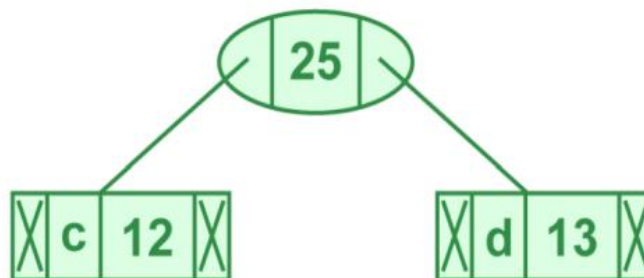
Step 1:

- Find the two nodes with the lowest frequencies: **A (5)** and **B (9)**.
- Merge them into a new node **(AB)** with a combined frequency of **14**.



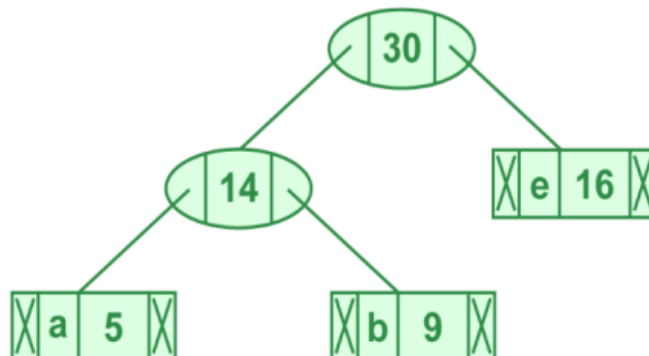
Step 2:

- Find the next two lowest frequency nodes: **C (12)** and **D (13)**.
- Combine them into a new node **(CD)** with a total frequency of **25**.



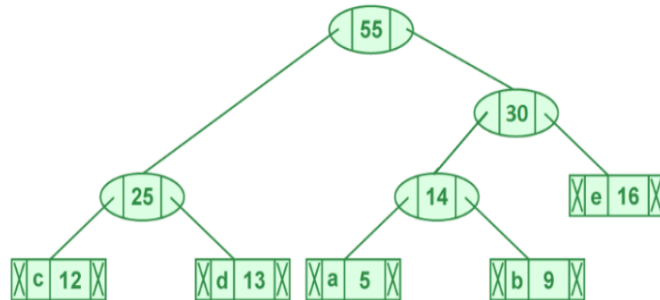
Step 3:

- Select the two lowest frequency nodes from the updated set: **(AB) (14)** and **E (16)**.
- Merge them into a new node **(ABE)** with a total frequency of **30**.



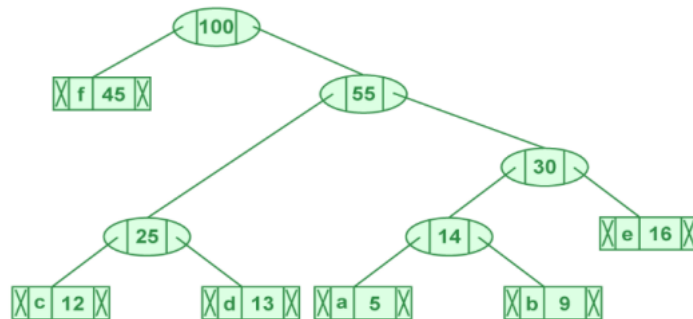
Step 4:

- Combine the nodes **(CD) (25)** and **(ABE) (30)** to form a new node **(CDABE) (55)** with a total frequency of 55.



Final Step:

- Combine **(CDABE) (55)** with the remaining node **F (45)** to form the **root node** with a total frequency of **100**.



Steps for Huffman Decompression:

- Reconstruct the Huffman Tree:**
 - If the tree structure or code table is stored along with the compressed data, we can use it directly.
 - Otherwise, we need to rebuild the tree using character frequencies or by analyzing the encoded bitstream.
- Decode the Compressed Data:**
 - Start at the **root** of the Huffman tree.
 - Read the bits from the encoded data one by one:
 - Move **left** when encountering "0".
 - Move **right** when encountering "1".
 - When you reach a **leaf node**, it represents a character—output it and go back to the root.
 - Repeat this process until all bits have been processed.
- Reconstruct the Original Message:**
 - As you decode each character, piece them together to rebuild the original text.

Example:

Let's say we have a Huffman tree and the encoded bitstream: 1100.

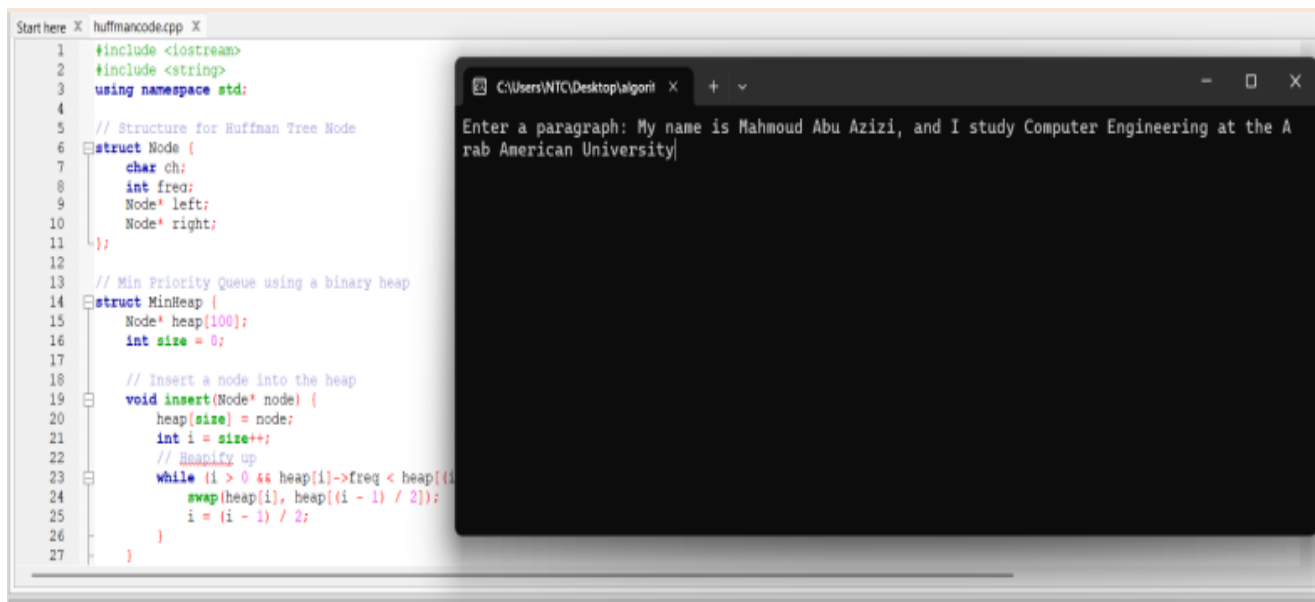
1. Start at the **root** of the tree:
 - o 1 → Move **right**
 - o 1 → Move **right**
 - o 0 → Move **left**
 - o 0 → Move **left** → (Leaf Node: 'a')
2. Output 'a' and return to the root.
3. Continue decoding the remaining bits in the same way.

By following steps, Huffman decompression efficiently restores the original data. This method is widely used in **file compression, network data transfer, and multimedia storage**, where preserving data integrity while reducing file size is essential.

CODE(C++) AND PSEUDOCODE:

<https://github.com/zaidardah/Huffman>

Examples Using Code Blocks Compiler and online compiler:



```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Structure for Huffman Tree Node
6 struct Node {
7     char ch;
8     int freq;
9     Node* left;
10    Node* right;
11 };
12
13 // Min Priority Queue using a binary heap
14 struct MinHeap {
15     Node* heap[100];
16     int size = 0;
17
18     // Insert a node into the heap
19     void insert(Node* node) {
20         heap[size] = node;
21         int i = size++;
22         // Heapify up
23         while (i > 0 && heap[i]->freq < heap[(i-1)/2]->freq) {
24             swap(heap[i], heap[(i-1)/2]);
25             i = (i-1)/2;
26         }
27     }
28 }
```

```
Enter a paragraph: My name is Mahmoud Abu Azizi, and I study Computer Engineering at the Arab American University
```

```

Start here x huffmancode.cpp x
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Structure for Huffman Tree Node
6 struct Node {
7     char ch;
8     int freq;
9     Node* left;
10    Node* right;
11 };
12
13 // Min Priority Queue using a binary heap
14 struct MinHeap {
15     Node* heap[100];
16     int size = 0;
17
18     // Insert a node into the heap
19     void insert(Node* node) {
20         heap[size] = node;
21         int i = size++;
22         // Heapify up
23         while (i > 0 && heap[i->freq < heap[i/2->freq]) {
24             swap(heap[i], heap[i/2]);
25             i = (i - 1) / 2;
26         }
27     }

```

```

C:\Users\NTC\Desktop\algori x + v

Encoded paragraph in Huffman code: 11100101110110111000101011001110000010111101110011000
0101010101111111111111010100110001011101011110110001000110000011000011101101101000101110100
1101110000110010110100111101010001110110111111111111101010011101110010010010110110011111
11011111110000101110011001011000010111111011010000100110010001010100111000100110100011101
011000101010110010110000001111100010110111100111101100011100011001011001011000010001110
Total bits required: 415

Enter a string to encode using Huffman codes: mh
Encoded String: 1010101010

Enter a binary string to decode: 1110010111011011100010101100111000001011110111001100001
01010101111111111111010100110001011101011110110001000110000001100001110110110100010111010011
01110000110010110100111101010001110110111111111111101010011101111001001001011011001111111
011111110000101110011001011000010111111101101000010011001000101010011100010011010001110101
10001010110101100000011111000101101111001111101100011100011001011001011000010001110
Decoded String: My name is Mahmoud Abu Azizi, and I study Computer Engineering at the Arab
American University

```

```

Enter a paragraph: my name is zaid and I am a computer engineer

*** Huffman Code Table ***
-> ' ' (Freq: 9, Bits: 2): 00
-> 'I' (Freq: 1, Bits: 6): 110000
-> 'a' (Freq: 5, Bits: 3): 100
-> 'c' (Freq: 1, Bits: 6): 110001
-> 'd' (Freq: 2, Bits: 4): 0101
-> 'e' (Freq: 5, Bits: 3): 101
-> 'g' (Freq: 1, Bits: 5): 01101
-> 'i' (Freq: 3, Bits: 4): 0111
-> 'm' (Freq: 4, Bits: 4): 1110
-> 'n' (Freq: 4, Bits: 4): 1101
-> 'o' (Freq: 1, Bits: 6): 111101
-> 'p' (Freq: 1, Bits: 6): 110010
-> 'r' (Freq: 2, Bits: 5): 11111
-> 's' (Freq: 1, Bits: 6): 110011
-> 't' (Freq: 1, Bits: 5): 01100
-> 'u' (Freq: 1, Bits: 5): 01000
-> 'y' (Freq: 1, Bits: 5): 01001
-> 'z' (Freq: 1, Bits: 6): 111100

Encoded paragraph in Huffman code: 1110010010011011001110101000111110011001111001000111010100100110101010011000000100111000100001100011111
101111011001001000011001011111100101110101101011111011011011111
Total bits required: 166

Enter a string to encode using Huffman codes: zaid
Encoded String: 11110010001110101

```

```

Enter a binary string to decode: 11100100100110110011101010001111100110011110010001110101001001101010100110000001001110001000011000111110
111101100100100001100101111100101110101101011111011011111
Decoded String: my name is zaid and I am a computer engineer

```

Analysis and Evaluation:

Time Complexity:

1. Frequency Calculation (`count Frequencies`):

- **Input:** String of length n .
- **Process:** Iterate over each character and increment its count.
- **Time Complexity:** $O(n)$.

2. Building the Huffman Tree (`build Huffman Tree`):

Insertion into MinHeap:

- k unique characters, each inserted in $O(\log k)$.
- **Total:** $O(k \log k)$.

Merging Nodes:

- $k-1$ merge operations, each with two extractions and one insertion (each $O(\log k)$).
- **Total:** $O(k \log k)$.

Combined Complexity: $O(k \log k)$.

3. Generating Huffman Codes (`generate Codes`):

- **Process:** Traverse each node in the tree ($\approx 2k-1$ nodes).
- **Time Complexity:** $O(k)$.

4. Encoding (`encode`):

- **Process:** Process each character in the original text and append its code.
- **Time Complexity:** $O(nL)$, where L is the average code length (usually small).

5. Decoding (`decode`):

- **Process:** Process each bit of the encoded text (of length m) by traversing the tree.
- **Time Complexity:** $O(m)$.

Final Overall Complexity:

- **General Complexity:** $O(n \log k)$.
- **Practical Complexity:** Approximately $O(n)$ since the number of unique characters (k) is limited (e.g., 256).

Time complexity	The process
$O(n)$	Frequency calculation
$O(k \log k)$	Insert the contract into the stack
$O(k \log k)$	Building a Huffman Tree
$O(k)$	Create icons
$O(nL)$	Coding
$O(m)$	Decoding

Conclusion:

This project provides a comprehensive exploration of Huffman's algorithm and its impact on data compression. By combining theory with a hands-on implementation in C++, we showed how the algorithm effectively reduces file sizes without losing any information—especially when some characters appear more frequently than others. By giving shorter codes to common characters, Huffman encoding makes both storage and data transmission more efficient.

The results prove that Huffman's algorithm is a powerful and flexible compression method, useful for things like file storage, image compression, and video encoding. Its Greedy approach and structured design make it easy to scale and integrate into more advanced compression systems.

This study highlights the value of smart data processing techniques in saving storage space, cutting transmission costs, and improving overall efficiency. Looking ahead, combining Huffman encoding with other compression methods could make it even more effective for handling large amounts of data.

8. References and Sources:

1. Wikioedia-Data compression (https://en.wikipedia.org/wiki/Data_compression)
2. Wikipedia-ASCII(<https://en.wikipedia.org/wiki/ASCII>)
3. Wikipedia – Huffman coding(https://en.wikipedia.org/wiki/Huffman_coding)
4. GeeksforGeeks – Huffman coding(<https://www.geeksforgeeks.org/huffman-coding-greedy-algo-3/>)

THANK YOU